

You are a senior Python performance engineer specializing in Ollama, OCR pipelines, and document processing systems.

I have an invoice processing application that uses:

- Ollama
- Qwen 2.5 3B
- Tesseract OCR
- PDF invoices (typically 2 pages)
- Local machine: Intel Core Ultra 5 CPU, 16GB RAM, integrated graphics

Current problem:

Processing a single 2-page invoice takes approximately 2–3 minutes.

For this hardware and model size, I expect significantly faster processing.

Your task is to perform a COMPLETE PERFORMANCE AUDIT of the project and identify every bottleneck.

DO NOT immediately modify code.

FIRST:

1. Analyze the entire processing pipeline.
2. Trace execution flow.
3. Measure timing for every stage.
4. Generate a performance report.

Measure and report the execution time of:

- PDF loading
- PDF to image conversion
- Image preprocessing
- Tesseract OCR
- OCR post-processing
- Prompt construction
- Ollama request preparation
- Ollama inference
- JSON parsing
- Database operations
- Any additional processing steps

For every stage output:

- Execution time
- Percentage of total runtime
- Memory usage

- CPU usage (if possible)

Also detect:

1. Number of Ollama calls made per invoice
2. Number of OCR passes per page
3. Number of PDF renderings per page
4. Duplicate processing
5. Unnecessary loops
6. Blocking I/O operations
7. Repeated model loading
8. Repeated OCR initialization
9. Repeated PDF conversion
10. Excessive image resolutions
11. Large context windows
12. Oversized prompts
13. Redundant database queries
14. Unnecessary file writes
15. Memory leaks
16. Threading opportunities
17. Async opportunities

Specifically verify:

A. Ollama Usage

- Count every ollama.chat() call
- Count every ollama.generate() call
- Count every API request
- Identify repeated calls for extracting individual fields
- Determine whether multiple calls can be consolidated into one extraction call

B. OCR Pipeline

Check:

- Tesseract PSM mode
- Tesseract OEM mode
- Image DPI
- Image dimensions
- Image color depth
- Preprocessing steps

Report whether OCR is over-processing images.

C. Prompt Optimization

Measure:

- Prompt length
- OCR text length
- Token count
- Context size

Determine whether excessive prompt size is slowing inference.

D. Model Performance

Verify:

- Actual Ollama model being used
- Quantization level
- Model size
- Whether model is reloaded for every request
- Tokens per second during inference

Report:

- Prompt evaluation time
- Generation time
- Total inference time

E. PDF Processing

Check:

- PDF rendering library
- DPI settings
- Whether pages are rendered multiple times
- Whether rendered images are cached

F. Database

Check:

- Query count
- Query execution times
- Duplicate writes
- Unnecessary commits

AFTER ANALYSIS:

Create a report with:

1. Bottleneck ranking from highest impact to lowest impact.

2. Estimated time savings for each optimization.
3. Recommended fixes.
4. Code locations responsible.
5. Before/after performance estimates.

THEN implement optimizations in order of impact.

Optimization goals:

- Process a 2-page invoice in under 30 seconds if possible.
- Reduce Ollama calls to the minimum required.
- Reduce OCR processing time.
- Reduce prompt size.
- Avoid repeated work.
- Improve CPU utilization.
- Keep extraction accuracy unchanged.

Finally provide:

- Performance report
- Root cause analysis
- List of applied fixes
- Before vs after timing comparison
- Remaining bottlenecks